

8.2

MRSW integer register:

```
public class RegIntMRSWRegister implements Register<Integer> {
    RegBoolMRSWRegister[M] bit;

    public RegIntMRSWRegister() {
        bit[0].write(true);
        for (int i = 1; i < M; i++) {
            bit[i].write(false);
        }
    }

    public void write(int x) {
        bit[x].write(true);
        for (int i = x - 1; i >= 0; i--) {
            bit[i].write(false);
        }
    }

    public int read() {
        for (int i = 0; i < M; i++) {
            if (bit[i].read()) {
                return i;
            }
        }
    }
}
```

Lemma: Any call to `read` returns a value set by some `write` call.

Proof: At least one bit of the register is set at any moment in time, because:

- Before the register can be used there is a single `write` — the initial `write(0)`.
- Each call to `write` sets exactly one bit and resets only bits below the updated one, starting from the right. The former also implies that no out-of-thin-air values will appear.

Each call to `read` goes from left to right until it finds a set bit.

A reader will eventually find a set bit because:

- If it read a zero bit, then it will proceed to the right, and earlier we've shown that at least a single bit is always set
- Writers only reset bits that are to the left of the bit they set, and they only do it *after* setting the new bit.

□

Lemma: Described register is regular.

Proof: Regularity means that the register:

- returns the old value if there is no read-write overlap
- returns the old or one of the new values if there is an overlap.

From previous lemma we know that any `read` returns a value put by some `write`. Let's assume that the last non-overlapping `write` wrote value x .

If `read` returned x , there is no problem, it may have been any of the cases above.

If `read` returned a value not equal to x , then it was put by some `write`, and because the last non-overlapping `write` put an x , the new value must be from a concurrent `write`. □